R USERS GROUP · INVITED TALK · OCTOBER 2026

Agentic R

Using agentic systems to efficiently write, review and produce reproducible code



David E. Rebellón-Sánchez, MD, MSc, PhD(c)
London School of Hygiene & Tropical Medicine

HANDS-ON MATERIALS

github.com/derebellon/agentic-r-workshop

HOW WE GOT HERE

From autocomplete to coding agents

Help with writing code has climbed the same ladder, from guessing the next word to running whole analyses for you.



Autocomplete

The editor suggests the next token or function name, RStudio completion, IntelliSense. It guesses; it does not understand.

LLMs generate code

Copilot, ChatGPT. You describe it in words and get a function or a whole script, but it lives outside R, in a chat window.

LLMs inside R

Packages like **ellmer** (Posit) let you chat with any model **from R** and give it **tool calling**, so it runs your own R functions.

Agentic systems

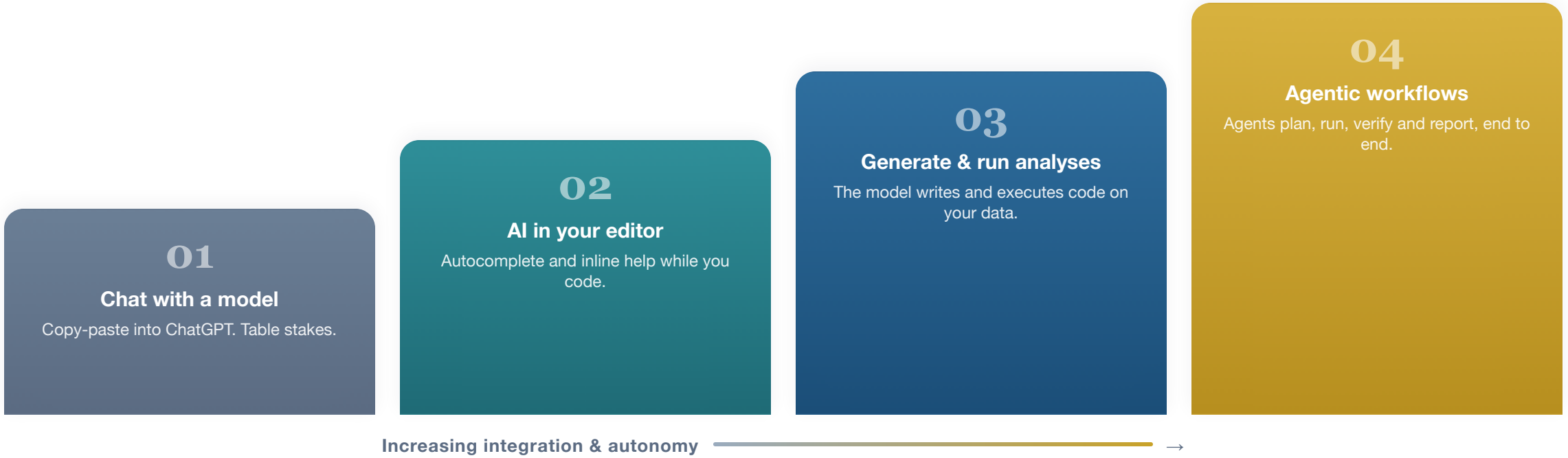
Claude Code and friends: agents that plan, run code, read the error, fix themselves and verify, a reproducible analysis end to end.

> The jump that matters for us: the model stops being a chat on the side and starts **acting inside your R session**, with your data and your functions.

FROM CHATTING TO DELEGATING

Four ways to work with AI

Not a ranking of people, a spectrum of how deeply AI fits into your work. Each step adds integration and autonomy, and all of them are useful.



AGENDA

What we will cover today

1. **What agentic AI is**
   From a model that answers to an agent that acts, uses tools and checks itself.

2. **A short history: from autocomplete to agents**
   How coding assistance evolved, and where R fits in.

3. **Agents inside R, and running them locally**
   ellmer, tool calling, and free local models with Ollama.

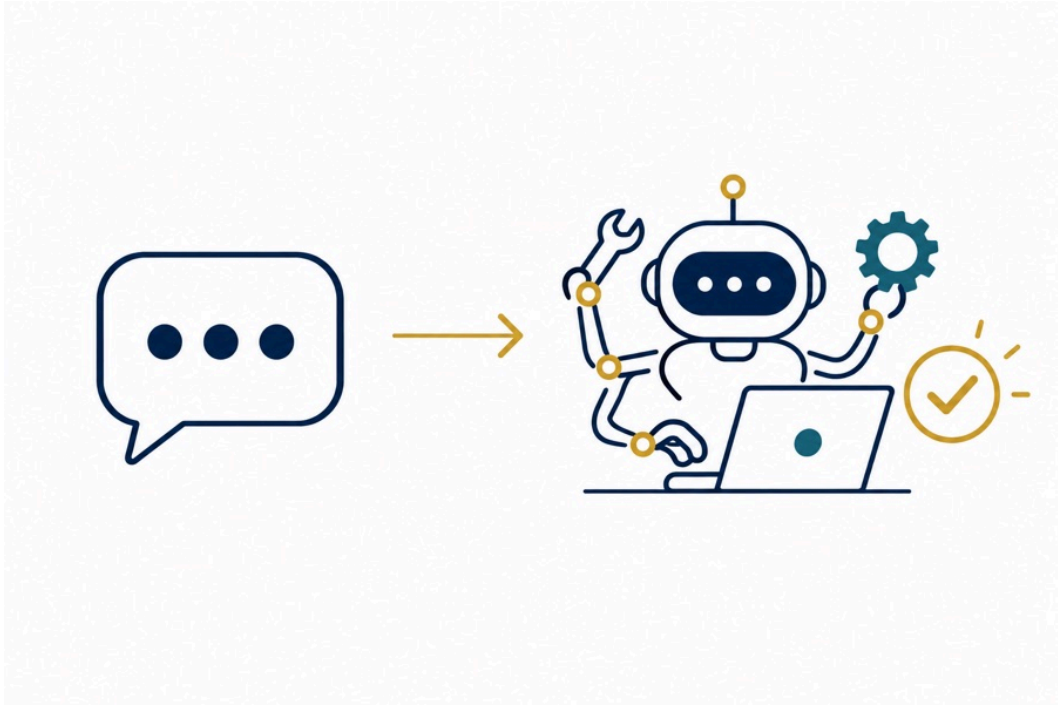
4. **Privacy with sensitive data**
   How to do all of this without your patient data ever leaving your machine.

5. **Live demo**
   An agent that builds a Table 1 and fits a model, on real-looking data.

PART 1

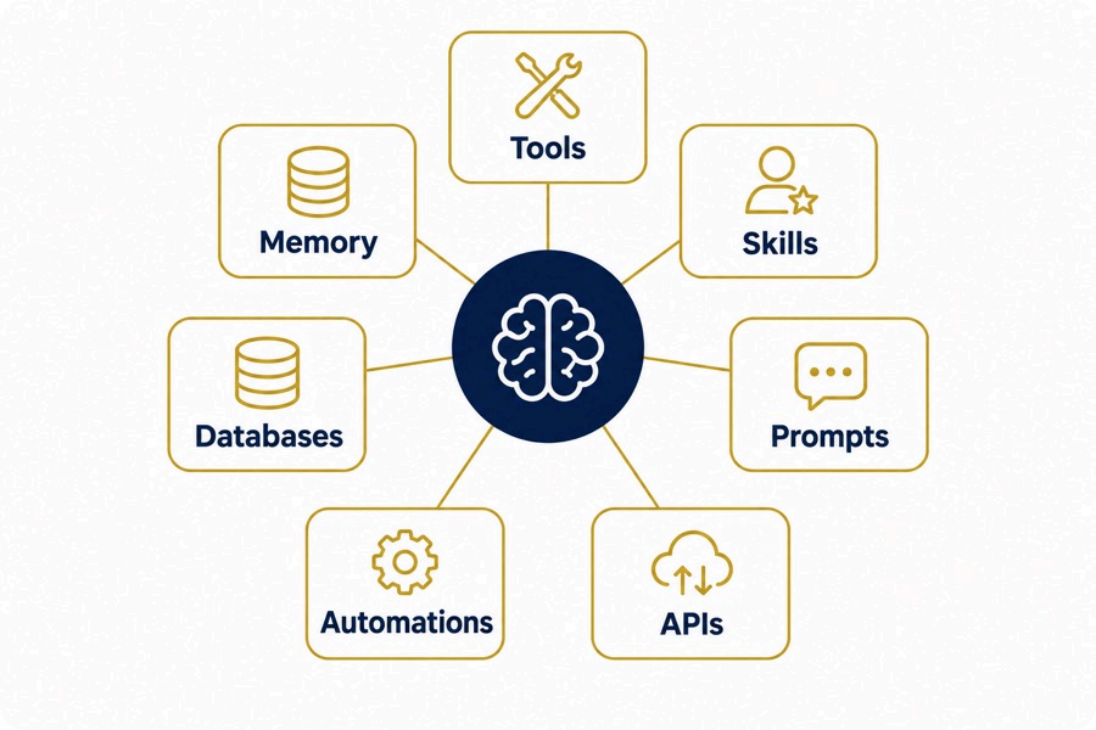
What agentic AI is

The shift from a model that answers to an agent that acts, uses tools and verifies itself.



FOUNDATIONS

Anatomy of an agent



- **Model**, the reasoning core. Open or closed, large or small, local or cloud.
- **Tools**, run your R code, read files, search literature, query databases.
- **Memory**, knows your project, your methods and your preferences.
- **Verification**, checks its own answers against the data and the output.
- **Human in the loop**, you approve, correct and decide. It accelerates; it does not replace your judgement.

WHAT IT DOES

What an agent does, day to day

Review email
Summarize, prioritize, draft replies.

Write code
R, Python, SAS: analysis and figures.

Organize repos
Structure projects in GitHub.

Generate reports
Reproducible, clear, professional.

Transcribe
Voice to structured text.

Review literature
Synthesis with verifiable citations.

Interpret omics
Genomics, proteomics, microbiome.

Draft science
Protocols, methods, discussion.

Autonomous agents
They watch and report unprompted.

BE HONEST ABOUT THE LIMITS

Risks, and how we manage them

Hallucination

Confident, plausible, sometimes wrong. → Verify every claim against the source.

Representation bias

Trained on uneven data; weaker on under-studied populations. → Validate against your gold standards.

Over-reliance

Skills atrophy if you stop thinking. → AI accelerates; your judgement still decides.

Reproducibility

Black-box steps are not science. → Log tools, parameters and versions; keep it auditable.

Data exposure

Sensitive data in the wrong channel. → Anonymize, run local, or no-retention by contract.

Cost & access

Not free, and unevenly distributed. → But local models are free; give access to the right people.

> None of these is a reason not to start. Each is **controllable by design**, and the biggest one, hallucination, is why a human stays in the loop.

HOW WE STAY HONEST

Build disagreement in, keep a human in charge

A model can **hallucinate**, and tell you the story you *want* to hear. So we make the system **argue with itself**, and keep a human in the loop.



Two agents, on purpose: a **worker** that runs the analysis, and an **opposer** that hunts biases and limits, "*what would make this wrong?*"



Then **you** check the findings against the literature and decide. The agent accelerates; **your judgement decides**.

> Ask the model for **two things**: the analysis you want, **and an adversary that tries to break it**. Agreement after a real fight beats a confident first answer.

FOUNDATIONS

Where can Agentic systems run?

You do not always need to send data to a public cloud. Think of it as a spectrum:

More control & privacy ——— **More convenience & power**

Local

On your computer · Ollama

- Runs on your machine
- Data never leaves · free · offline
- Great for short, routine tasks

Cloud

API / provider · Claude, GPT

- Runs in the provider's cloud
- Most powerful, long tasks
- Costs money · check retention

Key message: what matters is **where the model runs** and **what happens to your data**. The same R code works with either.

START SMALL

What an R-focused agent looks like



An agent focused on R could be tiny: a **model**, a few of your **R functions** as tools, and a small loop that ties them together, the **harness**.

FOUNDATIONS

The harness and the working loop

The **harness** is the software that turns a model into an agent that works in a loop until the task is done: it **perceives**, **reasons**, **uses a tool** (runs code, reads your files), then **verifies and reports**, or repeats.

Where does the harness live?

- **In an environment you own:** your own computer or server, with access only to the files you allow.
- **Sometimes you already use one:** every time you open the **Claude** or **ChatGPT** app, that is a harness running for you.

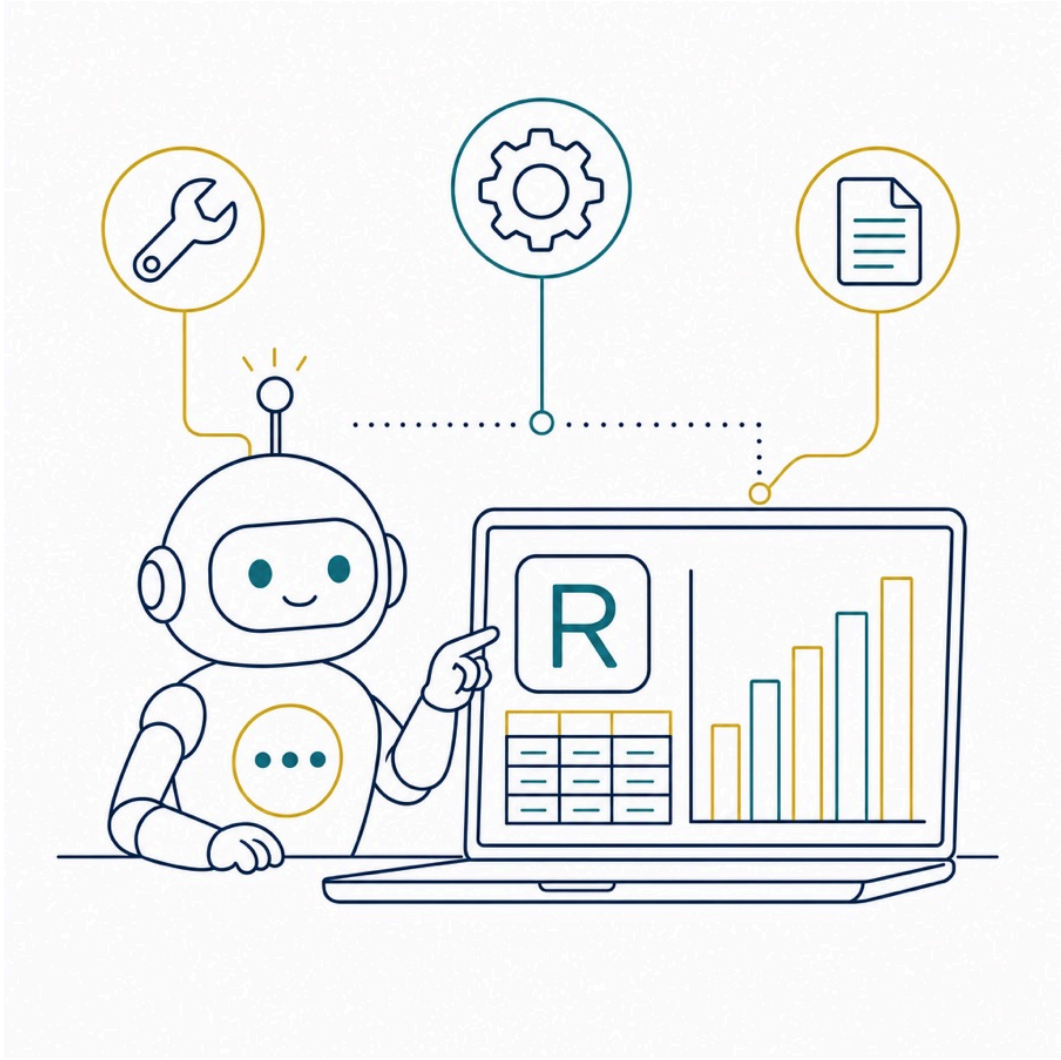
> In today's workshop the harness is about **50 lines of R**, so you can see every moving part.

```
# the loop, in words
repeat {
  perceive # read your request + state
  reason # decide the next step
  use_tool # run R, read a file
  verify # check the result
  if (done) report()
}
```

PART 2

Agents inside R

Chat with any model from R, give it your functions as tools, and run it on a free local model.



AGENTS INSIDE R

LLMs inside R: the ellmer package

ellmer (from Posit) is the bridge: it lets you talk to **any** model from R, local or cloud, with one API.

- **Tool calling:** register your own R functions, the model decides when to run them. That is what turns it into an agent.
- **Structured output:** extract text into a typed table (great for messy clinical notes).
- **Any engine:** a local model with **Ollama** (free, private) or Claude in the cloud, swapped in one line.

> You can build an agent right inside R. In practice, though, I prefer dedicated **external agents**, **Claude Code** or Google **Antigravity**, for heavier work; ellmer is ideal for embedding one in your R workflow.

```r
# chat with a LOCAL model, free
chat <- chat_ollama("qwen2.5:3b")

# give it one of your R functions
chat$register_tool(tool(
  make_table,
  "Build a Table 1 with gtsummary"
))

chat$chat("Table 1 by treatment")
#> [tool] make_table(by = "treatment")
```

AGENTS INSIDE R

Anatomy of a tool: how you teach it

A **tool** is how you give the agent hands and teach it **your** way. The model never sees your code, only the **description**, and from it decides **when** to call the function.

1. **The function**, real R that does the work (build the table with gtsummary).
2. **The description**, plain English: what it does and when to use it. This is what the model reads.
3. **The arguments**, the typed inputs it must fill in (e.g. which variable to compare by).

> Why bother? So it uses **gtsummary** (not invented code), returns **real numbers** (not hallucinations), and follows **your** conventions, reproducibly.

```r
# 1. the function, does the real work
make_table <- function(compare_by = NULL) {
  tbl_summary(data, by = compare_by) |> add_p()
}

# 2 + 3. wrap it: description + arguments
tool(
  make_table,
  "Build a Table 1 with gtsummary.
  Use compare_by to compare groups.",
  compare_by = type_string("variable to compare by")
)
```

PART 3

Privacy with sensitive data

How to do all of this without your patient data ever leaving your machine.



PRIVACY & COMPLIANCE

How your patient data stays private

Your data stays on your computer. If anything is shared, it is only **de-identified** results.



Your data
Patient records, stay on your computer

→



Local analysis
R on your machine

→



Results
Tables, models, figures

→



De-identify
Remove names & identifiers



Option 1 · Read locally

A local model (Ollama) interprets the results. **Nothing ever leaves your computer.**



Option 2 · Read in the cloud

Only de-identified results travel. With private models you can **turn off training and retention**; the safest is a **paid API with data-protection** terms.

Key message: the sensitive data stays local; only de-identified results may be shared. With a local model, even that stays home.

HANDS-ON

The workshop repo, step by step

What we'll do

1. **Download the repo**
   Scan the QR, or clone it from GitHub.

2. **Install the pieces**
   R, the ellmer package, and Ollama; then pull a model (qwen2.5:3b).

3. **Run the agent**
   In 02-complete-skeleton, ask: "Give me Table 1 by treatment, with p-values."

4. **Ask a real question**
   "Which variables raise mortality?", it fits a logistic regression, for real.

What's in the folder

```
agentic-r-workshop/
├─ data/  synthetic cohort (cohort.csv)
├─ 01-minimal-skeleton/  empty skeleton
├─ 02-complete-skeleton/  the agent "Rho"
├─ 03-claude-code/  same agent, Claude Code
└─ docs/  tutorial + these slides
```

github.com/derebellon/agentic-r-workshop

Thank you

Let's talk.

David E. Rebellón-Sánchez, MD, MSc, PhD(c)

London School of Hygiene & Tropical Medicine
github.com/derebellon/agentic-r-workshop